

MODULE 9 · TROUBLESHOOTING

Service Problems

Endpoints first — then ports, then names

"The service doesn't work" has a handful of causes

A Service is only as good as its EndpointSlice. When traffic doesn't get through, resist guessing — `kubectl get endpointslices` tells you in one command whether the problem is upstream (no pods selected) or downstream (routing, ports, names).

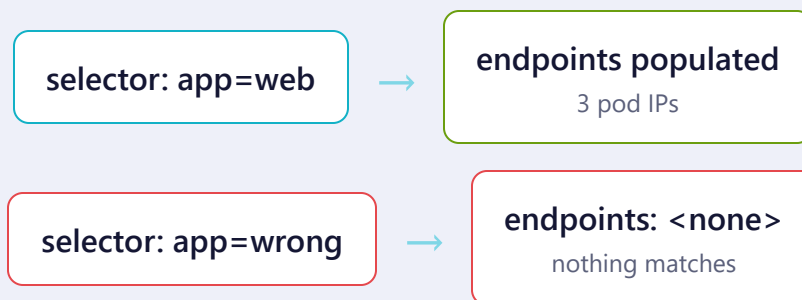
Four failure modes cover almost everything: **selector mismatch, port mismatch, wrong namespace, DNS.**

BY THE END YOU CAN

- Check endpoints before anything else
- Tell selector mismatch from port mismatch
- Fix cross-namespace and DNS lookups

Endpoints tell you which half is broken

SELECTOR → ENDPOINTS



Empty endpoints always means the selector or pod labels are wrong — never a routing or DNS problem. If endpoints ARE populated, the fault is further downstream: ports, namespace, or DNS.

What the symptom is telling you

SYMPTOM	LIKELY CAUSE	DIAGNOSTIC
<code>get endpointslices</code> shows empty <code>ENDPOINTS</code>	Selector / pod labels mismatch	Compare <code>svc.spec.selector</code> to pod labels
Endpoints present, connection refused	<code>targetPort</code> $\neq$ container's listening port	Compare endpoint port to <code>containerPort</code>
Endpoints present, intermittent failures	Some pods not Ready	<code>kubectl get pod</code> — <code>READY</code> column
Works via IP, not by name	Cross-namespace lookup, short name only	Use FQDN <code>svc.namespace.svc.cluster.local</code>
No DNS resolution at all	CoreDNS down / wrong nameserver	<code>nslookup</code> from a debug pod

One command before any other

- **get endpointslices first** — empty means selector/labels; populated means look downstream
- **compare selector to labels second** — only if endpoints came back empty

ORDER MATTERS

Don't curl-and-guess. EndpointSlice tells you in one command which half of the pipe is broken.

```
$ kubectl get endpointslices -n NS -l  
kubernetes.io/service-name=SVC  
# ENDPOINTS empty      = selector / label mismatch  
# ENDPOINTS has IPs    = selector OK, look downstream  
  
$ kubectl get svc SVC -o jsonpath='{.spec.selector}'  
$ kubectl get pods --show-labels
```

Zero endpoints, zero traffic

The service selects `app=wrong` but the pods carry `app=web`. The EndpointSlice controller finds nothing to add, the endpoints list stays empty, and every connection times out — kube-proxy has nowhere to send it.

Fix = correct the selector (or the pod labels) to match, then confirm endpoints populate.

```
$ kubectl get endpointslices -n training -l
kubernetes.io/service-name=svc-debug
NAME                ADDRESSTYPE  PORTS  ENDPOINTS
svc-debug-x7k2p     IPv4         80     <empty>

$ kubectl get svc svc-debug -o
jsonpath='{.spec.selector}'
{"app":"wrong"}
$ kubectl get pods --show-labels
app=web
```

Endpoints exist, but nothing answers

Endpoints inherit the service's `targetPort`, not the pod's actual listening port. Three healthy-looking IPs can still mean zero working connections — kube-proxy forwards faithfully, and the pod just resets the connection.

Fix = compare the endpoint port to `containerPort`, then patch `targetPort` to match.

```
Endpoints: 10.244.x.x:8080 (x3)
targetPort: 8080
containerPort: 80

$ kubectl patch svc SVC --type=json -p='[
{"op":"replace","path":"/spec/ports/0/targetPort","v
alue":80}]'
```

Where people trip

- **"Endpoints exist" ≠ "it works."** Still compare the port before declaring victory.
- **Curling the clusterIP first** instead of checking endpoints — wastes time guessing.
- **Forgetting the namespace.** Same short name in another namespace won't resolve without the FQDN.
- **Blaming the service for a DNS failure.** Run nslookup first to rule out CoreDNS.

Commands to keep at hand

```
$ kubectl get endpointslices -n NS -l kubernetes.io/service-name=SVC # first move – empty or populated?  
$ kubectl get svc SVC -o jsonpath='{.spec.selector}' # compare to pod labels  
$ kubectl get pods -l KEY=VAL --show-labels  
$ kubectl get svc SVC -o jsonpath='{.spec.ports[0].targetPort}'  
$ kubectl run dns-check --image=busybox:1.36 --rm -it --restart=Never -- nslookup SVC.NS.svc.cluster.local
```

Tip: endpoints → selector/labels → port → namespace/DNS. Check them in that order, every time.

RECAP

Service triage in one breath

- Empty endpoints = selector/label problem, not networking
- Endpoints present but broken = check targetPort vs containerPort
- Cross-namespace lookups need the FQDN
- DNS failures start with nslookup, not curl

HANDS-ON

Now run Lab 9.4
labs/9.4/

Next: [9.5 — DNS Problems](#)